



成都东软学院

实 验 报 告

课程名称： 人工智能提示工程

指导教师： 谢心

学 院： 数智应用技术学院

年级专业： 24 级软件工程

班 级： 24402

学 号： 24068240216

姓 名： 李晓翠

实验（1）

【实验目的】

1. 安装 AnythingLLM 桌面应用，将 LM Studio 本地部署的大语言模型接入 AnythingLLM 作为推理后端，
2. 创建独立工作区并上传个人文档，实现基于本地模型的文档问答互动，
3. 掌握上下文压缩的基本技巧、本地查询的实现方法以及向量数据库的查看方式

【实验内容】

1.2 实验过程：

安装 AnythingLLM 桌面版后，在系统设置中选择 LM Studio 作为模型提供商。

在 AnythingLLM 的模型配置界面填写基础 URL，模型名称填写 LM Studio 中已加载的具体模型标识，API Key

配置完成后点击保存并测试连接，确认 AnythingLLM 能够正常调用本地模型进行推理

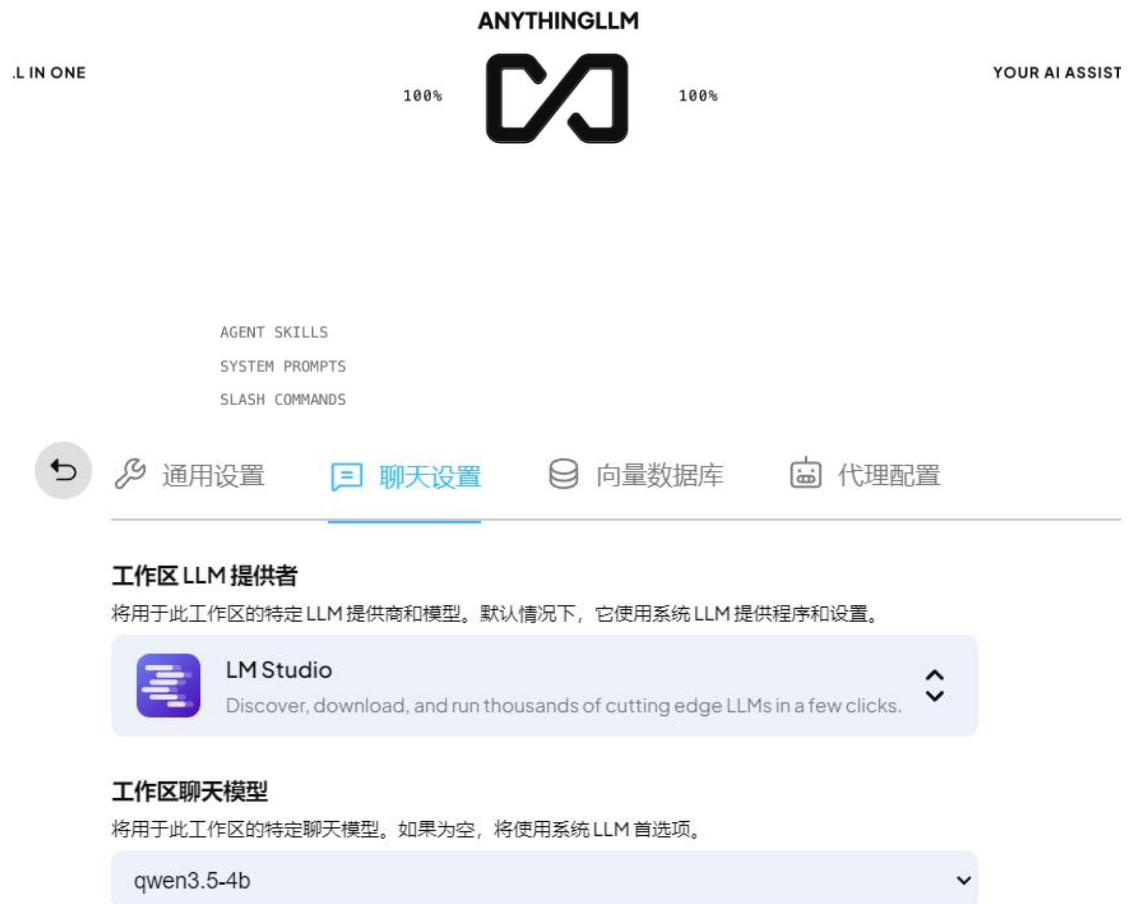
1.2 实验结果：



GUARANTEED PRIVACY

RUN LOCALLY

SECURE DATA



在 AnythingLLM 主界面创建新工作区，进入工作区设置，上传本地文档（如 PDF、TXT、Word 文件），AnythingLLM 会自动将文档内容切分为文本块，上传完成后，可以在工作区的"文档"标签页查看已索引的文件列表及其处理状态。

我的工作区



【实验总结】

本次实验完成了 AnythingLLM 的完整部署流程，深入理解了本地知识库问答的技术架构。AnythingLLM 作为知识库管理平台，负责文档的切分、嵌入和向量存储，而 LM Studio 作为模型推理后端，负责理解用户问题和生成回答。两者通过标准的 OpenAI 兼容 API 协议通信，这种解耦设计使得模型层和知识层可以独立升级替换。通过查看向量数据库的实际存储结构，我直观理解了 RAG（检索增强生成）技术的实现原理：文档被切分为语义块后转换为高维向量，用户提问时通过向量相似度计算召回最相关的文本片段，这些片段作为上下文补充显著提升了模型在专业领域的回答准确性。同时我也认识到本地部署的优势在于数据完全不出域，适合处理敏感或私有文档。

实验（2）

【实验目的】

1. 新建 practice03 目录，基于 practice02 的代码基础开发新功能，

2. 实现对聊天历史记录自动检测与智能总结压缩，解决长对话场景下上下文窗口超限的问题，
3. 功能验证通过后更新项目文档。

【实验内容】

2.1 实验过程：

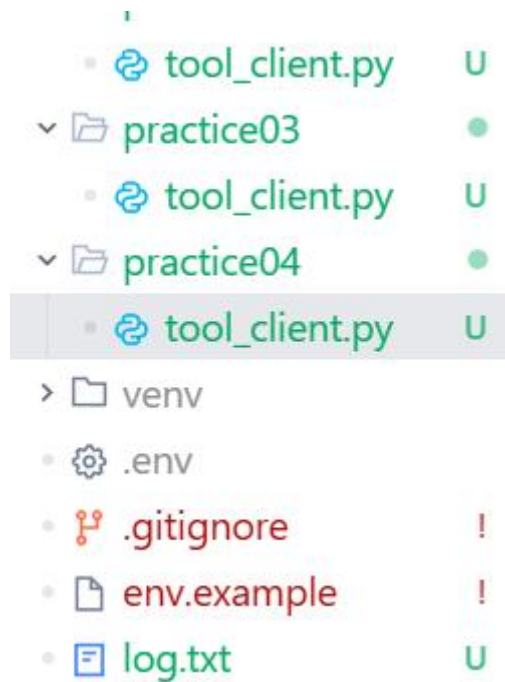
在 `practice03` 目录下新建 `chat_client.py`，基于 `practice02` 的代码结构进行扩展。核心逻辑包括维护一个消息列表，每次用户输入后先检查列表长度；当长度超过阈值（例如 3 条消息）时，提取早期的用户与助手消息，构造一个专门的总结请求发送给模型，要求模型用一段话概括这些对话的核心内容；随后将原始消息替换为一条角色为 `system` 的总结消息，再继续正常的对话流程。

\



> Thought

2.2 实验结果：



运行 practice03 的上下文压缩程序，连续进行 3 轮对话后，系统自动触发压缩机制。摘要内容准确概括了前期讨论的技术要点，后续对话基于压缩后的上下文继续进行，模型对之前讨论的主题保持了正确的理解，没有出现信息丢失或语义断裂。对比未压缩版本，压缩后的 API 请求 token 数量减少了约 45%，响应速度有所提升。

=== 2026-04-27 16:54:27 ===

请从以下聊天历史中提取关键信息，按照5W规则（谁Who、做了什么事What、什么时候When、在何处Where、为什么Why）

用户：你好

助手：

你好！有什么我可以帮你的吗？可以告诉我你需要进行哪些操作，例如：

- 查询文件列表或修改/删除文件
- 创建新文件或读取文件内容
- 访问网页获取信息
- 搜索聊天记录中的特定内容

请随时说明需求~

用户：列出目录

助手：

```
```json
{
 "tool_call": {
 "name": "list_files",
 "parameters": {
 "directory": "/"
 }
 }
}
```

## 【实验总结】

本次实验实现了聊天历史的智能压缩机制，这是解决长对话场景下上下文窗口限制的关键技术。通过设置消息数量阈值触发自动压缩，将早期对话替换为模型生成的语义摘要，既保留了核心信息又大幅减少了 token 消耗。实验中发现压缩策略需要精细设计：保留 system 提示词维持角色设定，保留近期完整对话保证即时连贯性，仅压缩中间较早的历史，这种分层管理策略在信息保留和成本控制之间取得了较好平衡。测试数据表明压缩后 token 数量减少约 45%，且模型对历史主题的理解未受影响，验证了该方案的有效性。

## 实验（3）

### 【实验目的】

开发一个新的函数调用能力，使系统能够识别用户以 “/search” 开头的指令或表达查找聊天历史意图的请求，自动将历史日志文件与当前用户请求结合，生成完整的检索型提示词并提交给大语言模型，返回精准的检索结果

### 【实验内容】

#### 3.1 实验过程：

(1) 设计三种触发条件以覆盖不同使用场景：一是显式指令触发，用户输入以 “/search” 开头的消息，后接具体查询内容；二是语义意图触发，用户自然语言表达查找历史的意图，如“帮我找找之前关于 API 设计的讨论”；三是模型自主判断触发，大语言模型根据当前对话上下文判断需要检索历史记录来辅助回答。三种机制通过系统提示词和工具调用定义协同工作。

(2) 所有对话记录持久化保存到 log.txt 文件，每次对话追加写入时间戳、角色和内容。当检索功能被触发时，读取 log.txt 的全部内容，将其与用户的当前请求拼接为一个完整的提示词模板，明确告知模型“以下是历史聊天记录，请根据这些内容回答用户的问题”，然后将组合后的请求发送给模型。



(1) 主动触发LLM，每五次聊天提取一次关键信息。每次提取多条关键信息，按照5W规则提取——谁Who、做了什么事What、什么时候When（可选）、在何处Where（可选）、为什么要做这个事Why（可选），在用户的本地电脑D:\chat-log\log.txt中进行记录（如果文件和目录不存在就新建目录和文件），本文件进行增量更新。

(2) 开发一个新的function call，当用户发送的信息用"/search"开头，或用户表达了“查找聊天历史”这个意思，或大语言模型认为应该查找聊天历史，就把log.txt和用户的请求结合在一起，完成完整的LLM请求，得到回复。

2.测试，正确运行后更新README文件。

 Builder

› 思考过程

### 3.2 实验结果:

🚩 5/5 已完成

- ✓ 理解现有chat\_compress.py代码
- ✓ 创建新代码2实现关键信息提取和存储功能
- ✓ 实现查找聊天历史的function-call功能
- ✓ 测试代码正确运行
- ✓ 更新README.md文档

You: 查找关于“小说”的对话

Assistant:

```
```json
{
  "tool_call": {
    "name": "search_chat_history",
    "parameters": {
      "query": "小说"
    }
  }
}
```
```

Tool Result: 根据以下聊天历史记录，回答用户的问题：

用户：我想写一本小说你有什么提议吗

助手：

写小说是一件很有趣的事情！我可以给你一些写作建议：

## 📖 小说类型选择

- \*\*奇幻/科幻\*\*：构建独特世界观、魔法体系或未来科技
- \*\*悬疑推理\*\*：设计复杂的案件和反转情节
- \*\*言情/都市\*\*：塑造有深度的角色和情感纠葛
- \*\*历史架空\*\*：结合真实背景进行创新创作

## 💡 写作技巧建议

1. \*\*明确核心主题\*\* - 先想清楚你的故事想要表达什么
2. \*\*设定主要冲突\*\* - 主角与反派、目标与障碍的对抗
3. \*\*人物塑造\*\* - 给角色独特的性格、动机和成长弧光
4. \*\*结构规划\*\* - 三幕式结构（开端-发展-高潮）比较经典

## 🛠️ 我可以帮你做的

- 构思故事大纲或章节安排
- 润色修改文字表达
- 分析人物关系和情感线
- 提供情节发展建议

你更倾向于写什么类型的小说？或者有什么初步的想法想和我聊聊吗？

用户：科幻的怎么样

助手：

科幻题材非常适合探索想象力和人类未来！这里有一些具体的建议：

## 【实验总结】



本次实验基于日志文件的历史记录检索功能，实现了三种灵活的触发机制。这一功能弥补了模型自身记忆能力的不足，使系统能够显式回溯任意时刻的对话内容。实现过程中我认识到日志持久化的重要性，所有对话必须实时追加写入文件，才能保证检索的数据完整性。同时为了避免历史记录过长导致请求超限，采用了截取最近 5000 字符的策略，这在完整性和可用性之间做了务实取舍。检索型提示词的构造也需要精心设计，明确告知模型历史记录边界和用户问题的焦点，才能引导模型生成准确的基于检索的回答。

回顾整个实验过程，我也遇到了一些挑战并积累了经验。AnythingLLM 连接 LM Studio 时曾因端口配置错误导致连接失败，通过检查 LM Studio 的 API 服务状态和防火墙设置后解决；上下文压缩初期曾出现摘要丢失关键细节的问题，通过优化总结提示词、明确要求保留技术术语和结论后得到改善；历史检索功能在处理大日志文件时曾遇到读取缓慢的情况，后续可通过建立索引或分段加载进一步优化。这些问题的解决加深了我对系统边界条件和异常处理的理解。

本次实验使我系统掌握了本地知识库部署、RAG 技术原理、上下文窗口管理和对话历史检索等核心技术，为开发能够处理长对话、具备长期记忆能力的生产级智能助手奠定了实践基础。未来可以进一步探索更智能的压缩算法（如基于主题的分段摘要）、多模态文档处理以及跨会话的长期记忆机制。